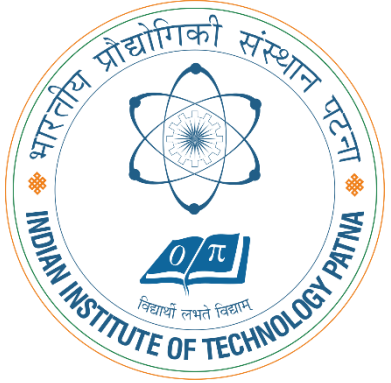


CS5102: FOUNDATIONS OF COMPUTER SYSTEMS

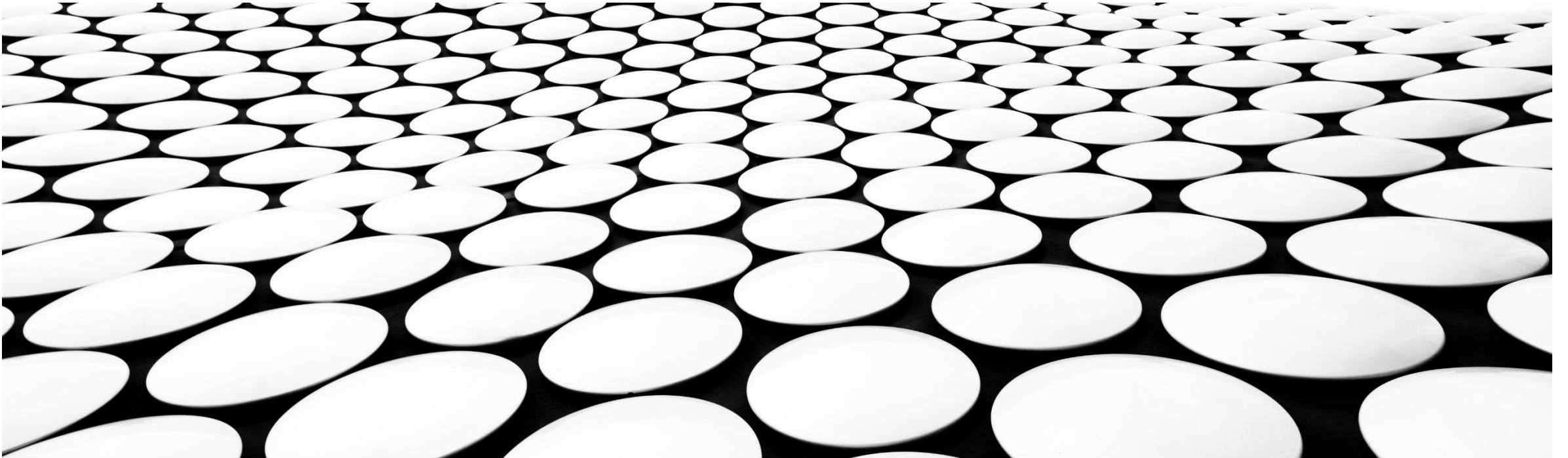


LECTURE 21: PROCESS SYNCHRONIZATION - II

DR. ARIJIT ROY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY PATNA





RECAP

RACE CONDITION



Process type

Independent: Do not affect other processes

Co-operative: Affects other processes

Two processes P_i and P_j

✓ // For Process P_i	✓ // For Process P_j
<u>int a = Shared;</u>	<u>int b = Shared;</u>
✓ <u>a++;</u>	✓ <u>b--;</u>
Sleep(1);	Sleep(1);
<u>Shared = a;</u> ✓	<u>Shared = b;</u> ✓

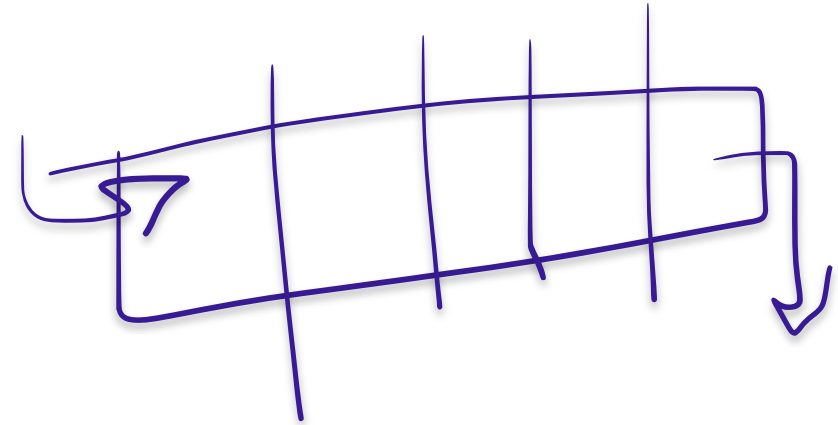
Race Condition

PRODUCER

Illustration of the problem:

Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers. We can do so by having an integer **counter** that keeps track of the number of full buffers. Initially, **counter** is set to 0. It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.

```
while (true) {  
    /* produce an item in next produced */  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```



CONSUMER



```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in next consumed */  
}
```

RACE CONDITION



- `counter++` could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- `counter--` could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with “count = 5” initially:

| S0: producer execute `register1 = counter`
| S1: producer execute `register1 = register1 + 1`
| S2: consumer execute `register2 = counter`
| S3: consumer execute `register2 = register2 - 1`
S4: producer execute `counter = register1`
S5: consumer execute `counter = register2`

{register1 = 5} ✓

{register1 = 6} ✓

{register2 = 5} ✓

{register2 = 4} ✓

{counter = 6} |

{counter = 4} |

CRITICAL SECTION PROBLEM

- Consider system of n processes $\{p_0, p_1, \dots, p_{n-1}\}$
- Each process has **critical section** segment of code
 - Process may be changing common variables, updating table, writing file, etc
 - When one process in critical section, no other may be in its critical section
- ***Critical section problem*** is to design protocol to solve this
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**

CRITICAL SECTION

- General structure of process P_i

```
do {  
    entry section ✓  
    critical section ✓  
    exit section ✓  
    remainder section ✓  
} while (true);
```

ALGORITHM FOR PROCESS P_i

do {

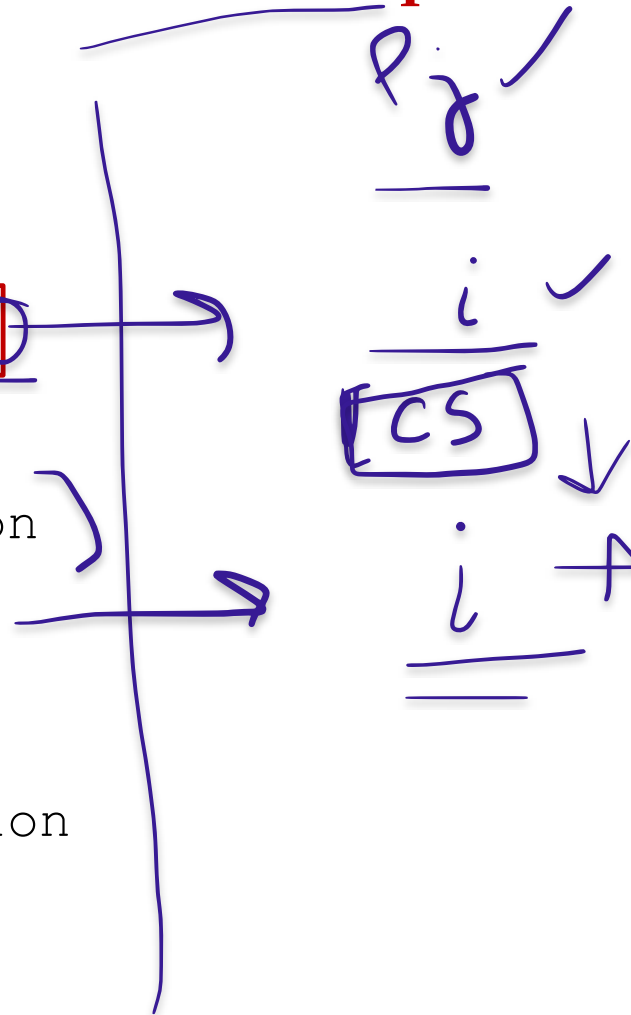
while (turn == j);

{ critical section }

turn = j;

remainder section

} while (true);



do {

entry section

critical section

exit section

remainder section

} while (true);

SOLUTION TO CRITICAL-SECTION PROBLEM

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
 - Assume that each process executes at a nonzero speed
 - No assumption concerning **relative speed** of the n processes

CRITICAL-SECTION HANDLING IN OS



Two approaches depending on if kernel is preemptive or non-preemptive

- **Preemptive** – allows preemption of process when running in kernel mode
- **Non-preemptive** – runs until exits kernel mode, blocks, or voluntarily yields CPU
 - Essentially free of race conditions in kernel mode

SOLUTION TO CRITICAL-SECTION PROBLEM USING LOCKS



```
do {  
    ✓ acquire lock ✓  
        critical section }  
    ✓ release lock ✓  
        remainder section  
} while (TRUE);
```

MUTEX LOCKS

- OS designers build software tools to solve critical section problem
- Simplest is mutex lock
- Protect a critical section by first **acquire()** a lock then **release()** the lock
 - Boolean variable indicating if lock is available or not
- Calls to **acquire()** and **release()** must be atomic
 - Usually implemented via hardware atomic instructions
- But this solution requires **busy waiting**
 - This lock therefore called a **spinlock**

ACQUIRE() AND RELEASE()

```
acquire()  
{  
  while (!available)  
    ; /* busy wait */  
  available = false;  
}
```

```
release()  
{  
  available = true;  
}
```

do {
 acquire lock
 critical section
 release lock
 remainder section
} while (true);

In a simplified way

Step 1: while(lock == 1);
Step 2: Lock = 1;
Step 3: CS
Step 4: Lock = 0

Does it ensure mutual exclusion?



TEST AND SET

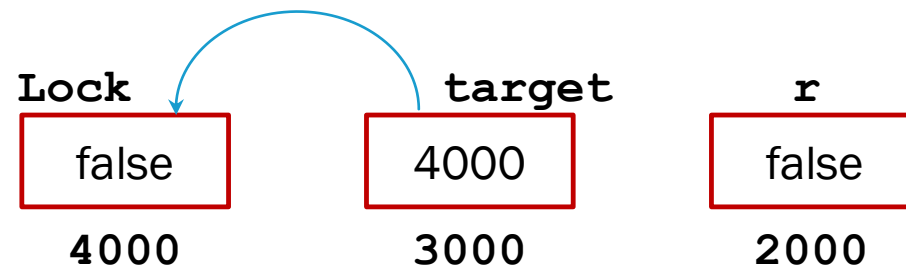
```
acquire()  
{  
  while(test_and_set(&Lock))  
    ; /* busy wait */  
}
```

Critical Section

Lock= false

}

```
boolean test_and_set(boolean *target)  
{  
  boolean r = *target;  
  *target = true;  
  return r;  
}
```



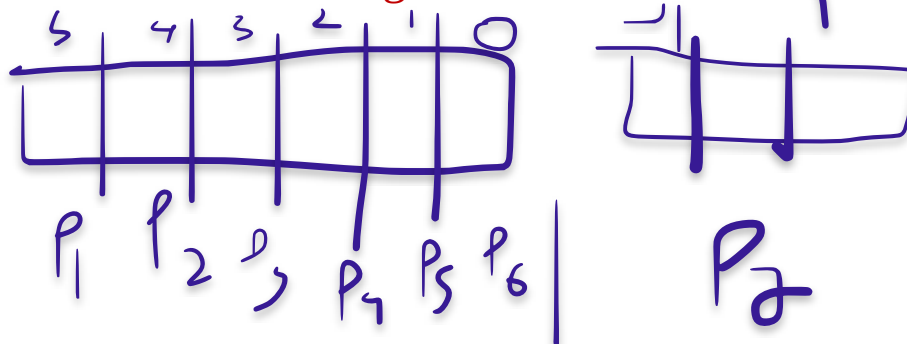


SEMAPHORE

SEMAPHORE

- Synchronization tool that provides more sophisticated ways (than Mutex locks) for process to synchronize their activities.
- Semaphore S – integer variable
- Can only be accessed via two indivisible (atomic) operations
 - wait() and signal()
 - Originally called P() and V()

In Dutch, Proberen – test and verhogen – increment



$S = 3$

Definition of the **wait()** operation

wait(S) ✓

```
{
  while (S <= 0) ✓
    ; // busy wait ✓
  S--; ✓
}
```

$S = 1$ ✓
 $S = 0$

Definition of the **signal()** operation

signal(S)

```
{
  S++;
}
```

P_i

SEMAPHORE USAGE



- **Counting semaphore** – integer value can range over an unrestricted domain

- **Binary semaphore** – integer value can range only between 0 and 1

- Same as a **mutex lock**
- Can solve various synchronization problems
- Consider P_1 and P_2 that require S_1 to happen before S_2

Create a semaphore “**synch**” initialized to 0

P1:

$S_1;$

signal(synch);

P2:

wait(synch);

$S_2;$

- Can implement a counting semaphore S as a binary semaphore

- Counting semaphore can be used to control access to a given resource of a finite number of instances
- Semaphore is initialized with the number of resources available
- Each process wish to access the resource perform **wait()** operation on semaphore
- When a process release the resource , it performs a **signal()** operation
- When the count for the semaphore goes to 0, all the resources are being used

SEMAPHORE IMPLEMENTATION



- Must guarantee that no two processes can execute the wait() and signal() on the same semaphore at the same time
- Thus, the implementation becomes the critical section problem where the **wait** and **signal** code are placed in the critical section
 - Could now have **busy waiting** in critical section implementation
 - But implementation code is short
 - Little busy waiting if critical section rarely occupied
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution



SEMAPHORE IMPLEMENTATION WITH NO BUSY WAITING

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
 - value (of type integer)
 - pointer to next record in the list
- Two operations:
 - **block** – place the process invoking the operation on the appropriate waiting queue
 - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue

```
typedef struct{  
    int value;  
    struct process *list;  
} semaphore;
```



IMPLEMENTATION WITH NO BUSY WAITING (CONT.)

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
}  
  
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

Try it:

Let S -> value = 3

Processes: P1, P2, P3, P4, and P5

BINARY SEMAPHORE



```
p(semaphore S)
{
    if (S->value == 1) ✓
    {
        S->value = 0; ✓
    }
    else
    {
        Block the process and place in
        the suspend list, sleep();
    }
}
```

```
v(semaphore S)
{
    if (suspend list is empty)
    {
        S->value = 1;
    }
    else
    {
        Select a process from the
        suspend list, wakeup();
    }
}
```



CLASSICAL PROBLEMS OF SYNCHRONIZATION

- Classical problems used to test newly-proposed synchronization schemes
 - **Bounded-Buffer Problem**
 - Readers and Writers Problem
 - Dining-Philosophers Problem

BOUNDED-BUFFER PROBLEM

- n buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value n

Binary semaphore to **acquire** and **release** lock

Counting semaphore, whose initial value is the number of slots in the buffer are **full**

Counting semaphore, whose initial value is the number of slots in the buffer are **empty**

BOUNDED BUFFER PROBLEM (CONT.)

- The structure of the producer process

```
do {  
    ...  
    /* produce an item in next_produced */  
    ...  
    wait(empty);      Wait until empty > 0 and then decrement empty  
    wait(mutex);      Acquire lock  
    ...  
    /* add next produced to the buffer */  
    ...  
    signal(mutex);    Release lock  
    signal(full);      Increment full  
} while (true);
```

BOUNDED BUFFER PROBLEM (CONT.)

- The structure of the consumer process

```
Do {  
    wait(full) ;           Wait until full > 0 and then decrement full  
    // wait(mutex) ;       Acquire lock  
    ...  
    /* remove an item from buffer to next_consumed  
*/  
    ...  
    / signal(mutex) ;      Release lock  
    / signal(empty) ;      Increment empty  
    ...  
    /* consume the item in next consumed */  
    ...  
} while (true);
```

BOUNDED BUFFER PROBLEM: EXAMPLE

```

...
wait(empty);
wait(mutex);
...
Buffer[in] = item p
in = (in+1)mod n;
...
signal(mutex);
signal(full);

```

```

...
wait(full);
wait(mutex);
...
item c = Buffer[out]
out = (out+1)mod n;
...
signal(mutex);
signal(empty);

```

full = 3 ✓
 empty = 5
 mutex = 1
 in = 3

0	A ✓
1	B ✓
2	C ✓
3	.
4	.
5	.
6	.
7	.

BOUNDED BUFFER PROBLEM: EXAMPLE

```

...
wait(empty);
wait(mutex);
...
Buffer[in] = item p
in = (in+1)mod n;
...

```

```

→ signal(mutex);
→ signal(full);

```

```

...
wait(full);
wait(mutex);
...
item c = Buffer[out]
out = (out+1)mod n;
...

```

```

signal(mutex);
signal(empty);

```

full = 3 ✓
 empty = 4 ✓
 mutex = 0 ✓
 in = 4 ✓

0	A
1	B
2	C
3	D ✓
4	
5	
6	
7	

BOUNDED BUFFER PROBLEM: EXAMPLE

```
...  
wait(empty);  
wait(mutex);  
  
...  
Buffer[in] = item p  
in = (in+1)mod n;  
...
```

```
signal(mutex);  
signal(full);
```

```
...  
wait(full);  
wait(mutex);  
  
...  
item c = Buffer[out]  
out = (out+1)mod n;  
...
```

```
signal(mutex);  
signal(empty);
```

full = 4
empty = 4
mutex = 1
in = 4

0	A
1	B
2	C
3	D
4	
5	
6	
7	

BOUNDED BUFFER PROBLEM: EXAMPLE



```
...  
wait(empty);  
wait(mutex);  
  
...  
Buffer[in] = item p  
in = (in+1)mod n;  
  
...  
signal(mutex);  
signal(full);
```

```
...  
wait(full);  
wait(mutex);  
  
...  
item c = Buffer[out]  
out = (out+1)mod n;  
  
...  
signal(mutex);  
signal(empty);
```

full = 4
empty = 4
mutex = 1
in = 4



0	A
1	B
2	C
3	D
4	
5	
6	
7	

BOUNDED BUFFER PROBLEM: EXAMPLE



```
...  
wait(empty);  
wait(mutex);  
  
...  
Buffer[in] = item p  
in = (in+1)mod n;  
  
...  
signal(mutex);  
signal(full);
```

```
...  
wait(full);  
wait(mutex);  
  
...  
item c = Buffer[out]  
out = (out+1)mod n;  
  
...  
signal(mutex);  
signal(empty);
```

full = 3 ✓
empty = 4 ✓
mutex = 0 ✓
in = 4

0	
1	B
2	C
3	D
4	
5	
6	
7	

BOUNDED BUFFER PROBLEM: EXAMPLE

```
...  
wait(empty);  
wait(mutex);  
  
...  
Buffer[in] = item p  
in = (in+1)mod n;  
  
...  
signal(mutex);  
signal(full);
```

```
...  
wait(full);  
wait(mutex);  
  
...  
item c = Buffer[out]  
out = (out+1)mod n;  
  
...  
signal(mutex);  
signal(empty);
```

full = 3
✓ empty = 5
✓ mutex = 1
in = 4

0	
1	B
2	C
3	D
4	
5	
6	
7	



BOUNDED BUFFER PROBLEM: CHECK FOR PREEMPTION

How it will be handled, if Producer process is preempted out after the execution of I1?

PI1

wait(empty);

PI2

wait(mutex);

...

Buffer[in] = item p

in = (in+1)mod n;

...

PI3

signal(mutex);

PI4

signal(full);

CI1

wait(full);

CI2

wait(mutex);

...

item c = Buffer[out]

out = (out+1)mod n;

...

CI3

signal(mutex);

CI4

signal(empty);

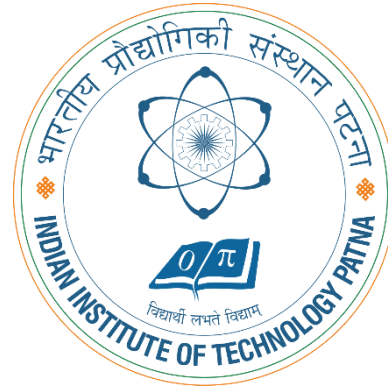
full = 3

empty = 5

mutex = 1

in = 3

0	A
1	B
2	C
3	
4	
5	
6	
7	



THANK YOU!